Moore serial two's complement

Read the input from the least-significant bit. Copy zeros and the first one, then invert every later bit. The three Moore states remember both where you are in that rule and which output bit should be visible.

Where this fits in your sheet

Entries 80. Day 5.

Entry numbers follow the tracker. Day labels in older filenames refer to when the notes were written; use the entry numbers to find the matching problem.

Pages	What to read
2	Why copy-through-first-one equals invert-plus-one
3-4	Why three states are needed and where every transition goes
5-7	Clock timing, complete code and the Mealy comparison
8	Mistakes, worked checks and recall questions

Your question

```
// explain me this code in a deep manner, along with need of each state
// ik the meaning and need of states s0, s1, s2 ... rest, explain me in doc
```

Reviewed against the saved tracker and available code on 7 September 2026. Earlier submission dates inside these notes are historical records.

1. What the serial machine computes

For a fixed-width binary word, two's complement means invert all bits and add one. That textbook operation looks global, but an LSB-first stream exposes a simpler local rule. The carry from the added one moves from low bits toward high bits, exactly in the same direction as the incoming stream.

Streaming rule

While the input is `0`, adding one keeps propagating, so output `0`. The first input `1` absorbs the carry and is copied as output `1`. Once that first `1` has appeared, the carry is finished, so every remaining input bit is inverted.

Why the rule is equivalent to invert-plus-one

- All trailing zeros below the least-significant `1` remain zero in the two's complement.
- The least-significant `1` itself remains one; it is the bit that terminates the add-one carry.
- Every more-significant bit above it is complemented.
- If the entire stream is zero, no first one ever occurs, and the output correctly remains all zero.

Worked 8-bit example

Take the parallel value `8'b0010_1100` (`8'h2C`). Its two's complement is `8'b1101_0100` (`8'hD4`). The serial order below is bit 0 first, so read each row from left to right in time.

Clock sample	0	1	2	3	4	5	6	7
Input x	0	0	1	1	0	1	0	0
Action	copy	copy	copy first 1	invert	invert	invert	invert	invert
Output z	0	0	1	0	1	0	1	1

Important direction detail

The method works online only because the least-significant bit arrives first. With an MSB-first stream, the circuit would not yet know where the least-significant one is, so it could not decide immediately which early bits to copy or invert.

2. Why a Moore machine needs three states

A state is not merely a label for the last input. It represents every piece of remembered history that can change present output or future behavior. Because this is a Moore machine, the output is a function of state alone: the input may choose the next state, but it cannot directly appear in the output equation.

Code state	Semantic name	What history it represents	Moore z
S0	COPY_ZERO	No 1 has been seen yet. The add-one carry is still conceptually propagating.	0
S1	OUTPUT_1	The bit just sampled must produce 1: either it was the first 1, or it was a later 0 being inverted.	1
S2	OUTPUT_0	A first 1 was already seen, and the bit just sampled was a later 1 being inverted.	0

Why S0 and S2 cannot be one state

Both states output `0`, but equal output does not make two states equivalent. Give both states the same next input `x=0`: from `S0`, that zero is still before the first one, so it must be copied and the next output stays `0`; from `S2`, the first one was already seen, so that zero must be inverted and the next output becomes `1`. Because the same future input requires different future behavior, the two histories must remain distinct.

Minimality argument

A Moore state partition must separate: (1) pre-first-one with output `0`, (2) post-first-one with output `1`, and (3) post-first-one with output `0`. Those are three distinguishable classes, so three Moore states are sufficient and, for this interface, necessary.

Why there is no S3

Two state bits provide four encodings, but the behavior needs only three states. `2'b11` is therefore unused. The `default` case sends any illegal or unknown encoding back to `S0`. The unused encoding is capacity, not evidence that a fourth behavioral state is missing.

A better way to name the states

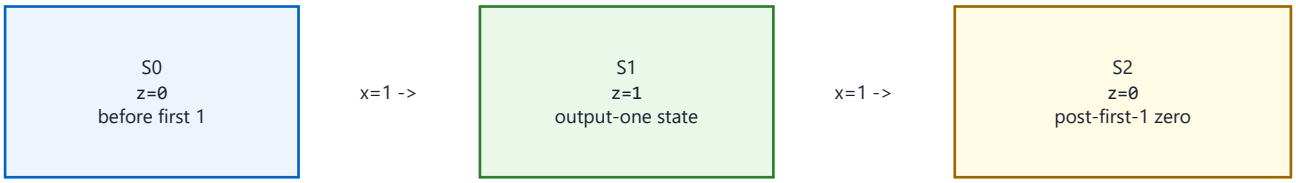
Names such as `COPY_ZERO`, `OUTPUT_1`, and `OUTPUT_0` expose the invariant directly. Numeric names are legal, but semantic names prevent the common mistake of treating `S1` as simply 'input was 1'. In fact, `S1` can be entered after a post-first-one input of zero because that zero must be inverted to one.

3. Every transition, decoded

The table should be read at a rising edge: start in the current state, sample x, choose next_state, update state, and then observe z from the new current state.

Current	x	Next	z after edge	Reason
S0	0	S0	0	Still before the first 1; copy the zero.
S0	1	S1	1	This is the first 1; copy it and enter inversion phase.
S1	0	S1	1	Already past the first 1; invert 0 to 1.
S1	1	S2	0	Already past the first 1; invert 1 to 0.
S2	0	S1	1	Already past the first 1; invert 0 to 1.
S2	1	S2	0	Already past the first 1; invert 1 to 0.

The complete state picture



Self-loops: `S0 --x=0--> S0`, `S1 --x=0--> S1`, and `S2 --x=1--> S2`. The cross-edge `S2 --x=0--> S1` completes the inversion rule.

The line that fixed the unsuccessful version

The successful code uses `S2: next_state = x ? S2 : S1;`. In the earlier non-success, the two destinations were swapped. After the first one, input `1` must produce output `0` and therefore go to `S2`; input `0` must produce output `1` and therefore go to `S1`.

4. Clock-by-clock timing trace

The example uses input `8'b0010_1100` in parallel notation, supplied serially as bit 0 through bit 7. Reset first places the machine in `S0`. Each row shows the state after that row's rising edge, which is also when the corresponding output bit becomes valid.

Edge	Sampled x	State before	State after	z after	Interpretation
reset	-	unknown	S0	0	Asynchronous reset establishes copy phase.
1	0	S0	S0	0	Copy a zero before the first one.
2	0	S0	S0	0	Copy another zero.
3	1	S0	S1	1	Copy the first one; inversion phase begins.
4	1	S1	S2	0	Invert later one to zero.
5	0	S2	S1	1	Invert later zero to one.
6	1	S1	S2	0	Invert later one to zero.
7	0	S2	S1	1	Invert later zero to one.
8	0	S1	S1	1	Invert later zero to one.

Is this Moore output one cycle late?

A Moore output is determined by the state that exists at the instant you observe it. Immediately before a rising edge, `z` still describes the previous state. At the edge, `x` is sampled and `state <= next_state` schedules the state update. After the nonblocking-assignment update, the continuous assignment recomputes `z` from the new state. HDLBits compares the output in this post-edge sense, so the output bit associated with the sampled input is available just after that edge.

> **Do not sample in the wrong phase**
>
> If a testbench checks `z` in the same simulation region as `@(posedge clk)` without allowing the nonblocking assignment to settle, it may see the previous value and falsely report a one-cycle error. Check after a small delay or in a later clocking region.

What asynchronous reset changes

Because the register block is sensitive to both `posedge clk` and `posedge areset`, asserting reset changes state to `S0` without waiting for a clock. Since `z` is decoded from state, it then returns to zero as well. Releasing reset does not itself trigger the block; normal conversion resumes on the next rising clock edge.

5. The successful code, block by block

```
module top_module (
    input clk, input areset, input x, output z
);
    reg [1:0] state, next_state;
    parameter S0 = 2'b00;
    parameter S1 = 2'b01;
    parameter S2 = 2'b10;

    always @(posedge clk or posedge areset) begin
        if (areset)
            state <= S0;
        else
            state <= next_state;
    end

    always @(*) begin
        next_state = state;
        case (state)
            S0: next_state = x ? S1 : S0;
            S1: next_state = x ? S2 : S1;
            S2: next_state = x ? S2 : S1;
            default: next_state = S0;
        endcase
    end

    assign z = (state == S1);
endmodule
```

Declarations and encoding

- `state` stores the current Moore state. `next_state` is purely combinational and becomes the next registered value.
- Three states require at least two state bits because `ceil(log2(3)) = 2`. The fourth binary code is unused.

State register

- The event control implements a positive-edge clock and positive-level asynchronous reset assertion: `@(posedge clk or posedge areset)`.
- Nonblocking assignment models simultaneous flip-flop updates and avoids procedural ordering artifacts.

Next-state block

- `always @(*)` automatically includes every read signal in the sensitivity list for combinational logic.
- `next_state = state` gives a safe hold default. Every legal state is then explicitly decoded, and `default` recovers from an unused encoding.

Output decode

- `assign z = (state == S1)` depends only on state, which is the defining Moore property. It is one only in the state whose semantic meaning is 'the current output bit must be one'.

6. Cleaner names and the Mealy contrast

Here is the same SystemVerilog design with names that describe each state:

```
module top_module (
    input  logic clk,
    input  logic areset,
    input  logic x,
    output logic z
);
    typedef enum logic [1:0] {
        COPY_ZERO = 2'b00,
        OUTPUT_1  = 2'b01,
        OUTPUT_0  = 2'b10
    } state_t;

    state_t state, next_state;

    always_ff @(posedge clk or posedge areset)
        if (areset) state <= COPY_ZERO;
        else        state <= next_state;

    always_comb begin
        case (state)
            COPY_ZERO: next_state = x ? OUTPUT_1 : COPY_ZERO;
            OUTPUT_1:  next_state = x ? OUTPUT_0 : OUTPUT_1;
            OUTPUT_0:  next_state = x ? OUTPUT_0 : OUTPUT_1;
            default:   next_state = COPY_ZERO;
        endcase
    end

    assign z = (state == OUTPUT_1);
endmodule
```

Why a Mealy solution needs only two states

A Mealy output may depend on both state and current input. It needs only a phase bit: `COPY` means no first one has appeared, with z=x; `INVERT` means the first one has appeared, with z=~x. The Moore version cannot put x directly in the output expression, so it must split the post-edge result into OUTPUT_1 and OUTPUT_0 states and also keep COPY_ZERO separate from the post-first-one zero state.

Architecture	Remembered states	Output rule	Key tradeoff
Moore	3	z = function(state)	More states; output changes only with state.
Mealy	2	z = function(state, x)	Fewer states; output has a direct combinational input path.

Do not mix the two designs accidentally

Your accepted entry 80 is Moore and your accepted entry 85 is Mealy. Both implement the same serial arithmetic rule, but their state meanings and output timing equations are intentionally different.

7. Mistakes, verification, and recall

Mistakes worth remembering

Mistake	Why it fails	Correct recall
Swap S2 destinations	It maps post-first-one input 1 to output 1 and input 0 to output 0.	In inversion phase: 1 -> S2/z0, 0 -> S1/z1.
Merge S0 and S2	They both output zero but react differently to a future zero.	Same output does not imply equivalent future behavior.
Treat S1 as last input was 1	S1 is also reached when a later input zero is inverted.	Name states by phase and required output, not by raw input.
Read z before NBA update	The testbench observes the previous state value.	Associate each sampled bit with z just after the corresponding edge.
Add an S3 because 2 bits allow it	Encoding capacity is not a behavioral requirement.	Three distinguishable Moore situations need three states.

Verification record

- The signed-in Chrome submission selector reports a fresh successful submission for entry 80 at 9/2/2026, 4:28:11 PM.
- The latest non-success at 9/2/2026, 4:18:12 PM used the incorrect swapped S2 transition; the successful version corrects it.
- The reusable Icarus Verilog self-check passed the Moore two's-complement behavior together with the other reviewed entries.
- The trace in this document reconstructs 8'h2C -> 8'hD4 in LSB-first order.

Five-question recall test

Recall question	Answer
What happens before the first 1?	Copy input bits unchanged.
What happens after the first 1?	Invert every later input bit.
Why are S0 and S2 separate?	They output 0 but have different future behavior for x=0.
Which state produces z=1?	Only S1 / OUTPUT_1.
What is the correct S2 transition?	x=1 stays S2; x=0 goes to S1.

Why the states are needed

S0 exists to remember that the first one has not appeared. S1 exists because a Moore machine needs a state whose output is one. S2 exists because, after the first one, output zero has different future behavior from the pre-first-one zero of S0. The transition table is exactly the serial two's-complement algorithm expressed as Moore state memory.

Original HDLBits entry 80 page: exams/ece241_2014_q5a